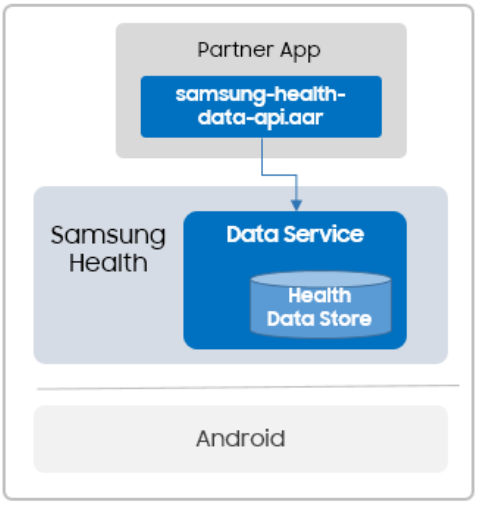
Programming Guide
Samsung Health Data SDK

v1.0.0 Beta1

The Samsung Health application has various features to measure user's health data. It allows the user to automatically record a range of activities with a Galaxy Watch and any connected accessories. The user can check their daily steps, heart rate, sleep, nutrition, and many more of their lifestyle metrics. Managing a healthy lifestyle is easier and simpler with Samsung Health.

Samsung Health Data SDK helps to access the health data in the Samsung Health application. An application importing the SDK library (`samsung-health-data-api.aar`) can read selected health data from Samsung Health's data store. Health data in the Samsung Health application may come from different sources, like the Galaxy Watch, which automatically transfers its data to a paired mobile phone's Samsung Health application.



Android Phone

Development Environment

Target Device

The Samsung Health Data SDK runs on devices with Android 10 (API level 29) or above. It is available on all Samsung smartphones and non-Samsung Android smartphones.

SDK Content

The SDK provides the following content:

Folder structure	Content description
`/docs`	Guide and API Reference
`/libs`	Import `samsung-health-data-api.aar` in libs of your project.
`/sample-code`	Sample application codes - HealthDiary
/tool	DataViewer

Limitations

- The Samsung Health Data SDK works with the Samsung Health application. Samsung Health version 6.28 or higher is required.
- The emulator is not supported.
- Data obtained using Samsung Health Data SDK is for fitness and wellness information only; it is not for the diagnosis or treatment of any medical condition.

Developer Support

If you experience issues while using the Samsung Health Data SDK, please visit the Samsung Developer site > Support > Developer Support. After a Samsung Account login, submit a query with an event log and a detailed issue description.

You can get the event log with the following instruction:

1. A Wi-Fi connection on your phone is recommended.
2. Go to the phone's Samsung Health > **Settings** > **About Samsung Health**.
3. Tap the Samsung Health logo quickly, more than 10 times until the "Send log report" button appears. Click the button.
4. Select an email application from the popup.
5. Save the email's attachment and copy the application information such as the Samsung Health application version and device model.
6. Share the obtained log report by attaching it to the support request.

Device Settings

Turn on the Phone's Developer Options

To develop and debug your application, enable the Developer options:

1. Go to the phone **Settings**.
2. Tap **About device** or **About phone**.
3. Tap **Software information**.
4. Tap **Build number** seven times.
5. Depending on your device and operating system, you may not need to enter your pattern, a PIN, or a password to enable the Developer options menu.
6. Go back to the phone's **Settings**. The **Developer options** menu will appear.

   Depending on your device, it may appear under **Settings** > **General** > **Developer options**.

Samsung Health's Developer Mode

The Samsung Health Data SDK works properly for registered partner applications in a Samsung Health system. The Health team registers the partner application information including an application package name and signature (sha256), with an available data type scope in the Samsung system. Moreover, the Samsung Health application allows the SDK's data access only if a running application package name and signature both matched with the registered information.

A registered application signature in the Samsung Health system should be from the application release key. If a partner application developer would like to test the application with a different application key, activate the Samsung Health's application Developer mode.

> **Note**
>
> The Samsung Health Developer mode is ONLY intended for testing or debugging your application. It is NOT for application users. Do not provide a Developer mode guide to application users.

You can activate Developer mode with the following steps:

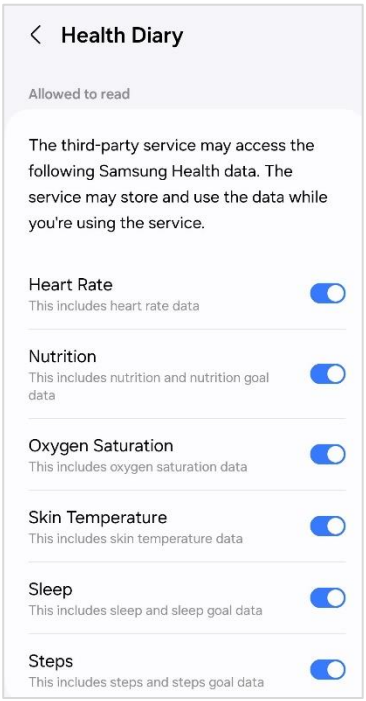
1. Select the ''more'' button of Samsung Health in the top-right.
2. Select **Settings** > **About Samsung Health**.
3. Tap the version line region quickly 10 times or more.
   If you are successful, the **Developer mode (new)** button is shown.
4. Select **Developer mode (new)** > **On**.

Main Features

Data Permission

Accessing the user's health data in Samsung Health is available after getting the user's consent. The Samsung Health Data SDK provides a data permission request feature. An application using the SDK can check the granted permissions and request permissions if more permissions are required.

Request for permissions is available with a set of permission list composed of a data type and access type. If `requestPermissions()` or `requestPermissionsAsync()` is called, the following data permission popup is shown. For example, developer will have access to heart rate data if the user of the application grants permission for **Heart Rate** in the **Allowed to read** section of the permission popup.



Applications using the Samsung Health Data SDK are listed up in Samsung Health > **Settings** > **Apps**.

> **Note**
>
> For the user data privacy, set a menu so that the user can change the data allowance state anytime in your application.

Data Types

The Samsung Health Data SDK supports various data types:

- Active summary
- Active calories burned goal
- Active time goal
- Blood glucose

- Blood oxygen
- Blood pressure
- Body composition
- Exercise
- Exercise location
- Floor climbed
- Heart rate
- Nutrition
- Nutrition goal
- Skin temperature
- Sleep
- Sleep goal
- Steps
- Step goal
- Water intake
- Water intake goal

User Profile Data

The user profile stores gender, date of birth, height, weight, and nickname information. It is available in read-only mode.

- User profile

Data Access

The Samsung Health Data SDK provides various operation types, allowing developers to access Samsung Health data. Available operations may vary, depending on the data type. Reading data, reading associated data and aggregating data gives a data point list, representing values stored in Samsung Health, such as a list of heart rate values from a specific time range.

Each data point includes a data ID and data source information representing the **application** and the **device**, which initially measured the data.

Read Data

A reading operation is a basic query to get a set of data from Samsung Health. Data types and conditions such as time range or data source can be set.

Read Associated Data

Reading associated data is useful to get multiple data types which are related to each other. For example, Samsung Health records oxygen saturation and skin temperature during the user's sleep. This operation helps to make a simple query to get sleep data (base data) with oxygen saturation and skin temperature data (associated data).

Aggregate Data

Aggregating data gives summarized and processed data that is frequently used like `TOTAL`, `MIN`, `MAX` and `LAST`. A data type, aggregating operation type and various other conditions can be applied while building an aggregate request.

Read Changes

Reading changes gives data change information for a given data type and conditions. It returns a change list. Each change item includes a change type as `UPSERT` and `DELETE` and a change time. In case of updated data, a data point is retrieved. For deleted data, it gives data ID. This feature helps keep the application current by reading only the changes made since the last synchronization with Samsung Health.

Device Manager

Saved health data in Samsung Health can be from various connected devices like a Galaxy Watch, Galaxy Ring and a weight scale. `DeviceManager` provides source device information of saved health data in the Samsung Health's data store.

For instance, when Galaxy Watch measures oxygen blood, the watch's measured data will be synchronized to a paired Android smartphone's Samsung Health. And if the user measures weight with a connected weight scale device, that weight data will be synchronized too. In this case, if the `DeviceManager.getDevices()` API is called with `DeviceGroup.WATCH`, the Galaxy Watch's `Device` object will be retrieved.

The `Device` object includes:

- Device ID
- Device type like `DeviceGroup.WATCH` or `DeviceGroup.ACCESSORY`
- Device's manufacturer name
- Device model name
- Device name costumed by the user

You can get device information for a specific device ID or a device type.

Filters

Filters help to limit a query range to get data that are more precise. The Samsung Health Data SDK provides time, source, and ID filters. Available filters are different for each data operation.

For instance:

- `IdFilter`, `ReadSourceFilter`, and `TimeFilter` are available for **reading data**.
- `AggregateSourceFilter` and `TimeFilter` are available for **aggregate data**.

Time Filters

A built request with a **start time** and **end time** gives all records that are inside this time range. For that, you can use the following filter types:

- `LocalTimeFilter`
- `LocalDateFilter`
- `InstantTimeFilter`

Source Filters

All health data stored in Samsung Heath has the following source information:

- Source application ID: an application package name which identifies the application that inserted the data to Samsung Health.
- Source device ID: representing a device which measures or creates the data.

`ReadSourceFilter` supports the following functions:

- `fromApplicationId`: an application package name
- `fromDeviceType`: a device type such as `MOBILE` or `WATCH`
- `fromLocalDevice`: a device with Samsung Health installed
- `fromPlatform`: measured or inserted by Samsung Health application

`AggregateSourceFilter` supports the `fromPlatform` function.

ID Filter

This filter is for setting a data ID.

Grouping Retrieved Data

Retrieved health data can be grouped into time segments with:

- `InstantTimeGroup`: `MINUTELY`, `HOURLY`, and `DAILY`
- `LocalDateGroup`: `DAILY`, `WEEKLY`, `MONTHLY`, and `YEARLY`
- `LocalTimeGroup`: `MINUTELY`, `HOURLY`, and `DAILY`

Hello Data SDK

This is a tutorial on how to use Samsung Health Data SDK.

Importing the SDK Library and Build Setup

Import the `samsung-health-data-api.aar` library into your application project's "libs" folder and add it into the module level `build.gradle` of your application project.

```
dependencies {

    implementation(fileTree(mapOf("dir" to "libs", "include" to listOf("*.aar"))))
    implementation "com.google.code.gson:gson:2.9.0"
}
```

Apply "kotlin-parcelize" plug-in to your module level `build.gradle` file.

```
plugins {

    id("kotlin-parcelize")
}
```

Permission Request

Granted Permission Check

Accessing health data requires the user's permissions for every data type. Check that the permissions are granted using:

- `HealthDataStore.getGrantedPermissions()`

It returns a set of granted permissions.

```
suspend fun checkForPermissions() {

    val healthDataStore = HealthDataService.getStore(context)
    val permSet = mutableSetOf(
        Permission.of(DataTypes.HEART_RATE, AccessType.READ),
        Permission.of(DataTypes.SLEEP_GOAL, AccessType.READ))

    val grantedPermissions = healthDataStore.getGrantedPermissions(permSet)
    if (grantedPermissions.containsAll(permSet)) {
        // All Permissions already granted
    } else {
        // Partial or No permission, we need to request for the permission popup
    }
}
```

Request for Permission

If permissions were not present, you need to request for permissions with:

- `HealthDataStore.requestPermissions()`

If this API succeeds, a data permission popup will display on the application activity with a list of permissions to grant by the user. After the user submits the permission popup, the API returns a set of granted permissions.

```
suspend fun requestForPermissions() {

    val healthDataStore = HealthDataService.getStore(context)
    val permSet = mutableSetOf(Permission.of(DataTypes.HEART_RATE, AccessType.READ),
        Permission.of(DataTypes.SLEEP_GOAL, AccessType.READ))

    val result = healthDataStore.requestPermissions(permSet, activity)
    if (result.containsAll(permSet)) {
        // All permissions granted
    } else {
        // Partial or no permission
    }
}
```

Read Data

You can now read some data from Samsung Health using:

- `HealthDataStore.readData()`

Apart from the data type, you can also set a time filter in the request.

The example below gives a data point list (representing heart rate values) from the specific time range.

```
suspend fun readHeartRateData(startDate: LocalDateTime, endDate: LocalDateTime) {

    val healthDataStore = HealthDataService.getStore(context)
    val localTimeFilter = LocalTimeFilter.of(startDate, endDate)
    val readRequest = DataTypes.HEART_RATE.readDataRequestBuilder
        .setLocalTimeFilter(localTimeFilter)
        .setOrdering(Ordering.DESC)
        .build()

    val heartRateList =
        healthDataStore.readData(readRequest).dataList
    // do data processing
}
```

The following example shows a way to get heart rate, measured by a watch (such as a Galaxy Watch paired with the mobile device):

```
val localTimeFilter = LocalTimeFilter.of(startDate, endDate)
val readRequest = DataTypes.HEART_RATE.readDataRequestBuilder
    .setSourceFilter(ReadSourceFilter.fromDeviceType(DeviceGroup.WATCH))
    .setLocalTimeFilter(localTimeFilter).setOrdering(Ordering.ASC)
    .build()
```

To get data created in the mobile device with Samsung Health installed, set a source filter with a local device.

```
.setSourceFilter(ReadSourceFilter.fromLocalDevice())
```

Aggregate Data Request

You can retrieve a data's total or minimum and maximum values or the last data entry by using the function:

- `HealthDataStore.aggregateData()`

Total Steps

To get total steps, use `DataType.StepsType.TOTAL`.

If a `LocalTimeFilter` is set from the beginning of today to the current time, the total aggregate operation's result is the same as step count shown in the Samsung Health's Pedometer tracker.

```
suspend fun readStepCount(startDate: LocalDateTime, endDate: LocalDateTime) {

    val healthDataStore = HealthDataService.getStore(context)
    val localtimeFilter = LocalTimeFilter.of(startDate, endDate)
    val readRequest = DataType.StepsType.TOTAL.requestBuilder
        .setLocalTimeFilter(localtimeFilter)
        .setOrdering(Ordering.ASC)
        .build()
    val dataList = healthDataStore.aggregateData(readRequest).dataList
    dataList.forEach {
        val stepCount = it.value
    }
}
```

Minimum and Maximum Heart Rate Values

You can retrieve minimum and maximum heart rate data from a specific time range using the example:

```
suspend fun minHeartRateAggregateRequest(startDate: LocalDate, endDate : LocalDate) {

    val healthDataStore = HealthDataService.getStore(context)
    val localDateFilter = LocalDateFilter.of(startDate, endDate)
    val minHeartRateAggregate = DataType.HeartRateType.MIN.requestBuilder
        .setLocalDateFilter(localDateFilter)
        .build()

    val dataList = healthDataStore.aggregateData(minHeartRateAggregate).dataList
    dataList.forEach {
        val minHR = it.value
    }
}

suspend fun maxHeartRateAggregateRequest(startDate: LocalDate, endDate : LocalDate) {

    val healthDataStore = HealthDataService.getStore(context)
    val localDateFilter = LocalDateFilter.of(startDate, endDate)
    val maxHeartRateAggregate = DataType.HeartRateType.MAX.requestBuilder
```

```
        .setLocalDateFilter(localDateFilter)
        .build()

    val dataList = healthDataStore.aggregateData(maxHeartRateAggregate).dataList
    dataList.forEach {
        val maxHR = it.value
    }
}
```

Last Bedtime of Sleep Goal Data

The aggregate operation LAST indicates last values saved or updated in Samsung Health.

To get the last bedtime goal, see the example below. This value is the same as that in Samsung Health > Sleep Tracker > Set target.

```
suspend fun lastBedTimeAggregateRequest(startDate: LocalDate,endDate : LocalDate) {

    val healthDataStore = HealthDataService.getStore(context)
    val localDateFilter = LocalDateFilter.of(startDate, endDate)
    val lastBedTimeAggregate = DataType.SleepGoalType.LAST_BED_TIME.requestBuilder
        .setLocalDateFilter(localDateFilter)
        .build()

    val dataList = healthDataStore.aggregateData(lastBedTimeAggregate).dataList
    dataList.forEach {
        val lastBedTime = it.value
    }
}
```

Read Data Changes

Health data in Samsung Health can be changed or deleted. If you need to get data changes for a specific data type, use:

- `HealthDataStore.readChanges()`

To read sleep data change, see the following example. If the user changed the sleep data in the Samsung Health application, the `readChanges()` gives a changed data result. The result's changed item includes a changed time, a change type with `ChangeType.UPSERT` and the updated data point. You can retrieve deleted sleep data using `ChangeType.DELETE`, it includes the ID of the deleted data record.

```
suspend fun getChangedSleepGoal(lastSyncTime: Instant, endTime: Instant) {

    val healthDataStore = HealthDataService.getStore(context)
    val changeRequest = DataTypes.SLEEP.changedDataRequestBuilder
        .setChangeTimeFilter(InstantTimeFilter.of(lastSyncTime, endTime))
        .build()

    val changes = healthDataStore.readChanges(changeRequest).dataList
    changes.forEach{ change ->
        if(change.changeType == ChangeType.DELETE) {
            val deletedDataUid = change.deleteDataUid
        } else if(change.changeType == ChangeType.UPSERT) {
            val upsertDataPoint = change.upsertDataPoint
        }
    }
}
```

Read Associated Data

You can read data for a specific data with associated data for one or more types.

- `HealthDataStore.readAssociateData()`

Samsung Health measures the user's oxygen saturation and skin temperature during sleep, if the user slept with a wearable device (like Galaxy Watch). Such data is present in Samsung Health's sleep tracker.

After getting a specific sleep data point, reading an associated oxygen saturation data is available as the following example.

```
private suspend fun readAssociatedOxygenSaturationData(sleepData: HealthDataPoint) {
    val healthDataStore = HealthDataService.getStore(context)
    val sleepSessionUid = sleepData.uid
    val readAssociateRequest = DataTypes.SLEEP.associatedReadRequestBuilder
        .setIdFilter(IdFilter.fromDataUid(sleepSessionUid))
        .addAssociatedDataType(DataType.SleepType.Associates.BLOOD_OXYGEN)
        .build()
    val oxygenList = healthDataStore.readAssociatedData(readAssociateRequest).dataList
    for (oxygenData in oxygenList) {
        val oxygenDataPoints = oxygenData.getDataPointOf(DataTypes.BLOOD_OXYGEN)
        oxygenDataPoints?.let {
            for (oxygenPoint in it) {
                val oxygen =
oxygenPoint.getValue(DataType.BloodOxygenType.OXYGEN_SATURATION)
            }
        }
    }
}
```

Devices Registered in Samsung Health

As stated above, all data has a device ID associated with it, meaning a device which is the source of the data (either measured by the device or created, for example, by manually inserting the data).

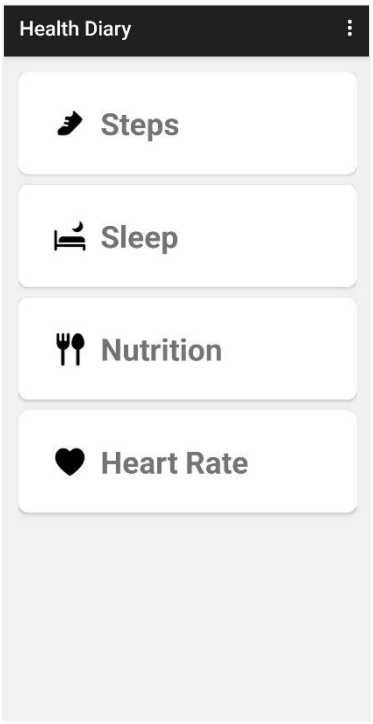
To get a list of the devices registered in Samsung Health, you should use DeviceManager. The example below shows how to get information about a local device (an Android device in which the Samsung Health application is installed) and a list of **watches** that have stored their data in Samsung Health:

```
val deviceManager = healthDataStore.getDeviceManager()

val localDevice = deviceManager.getLocalDevice()
Log.i(TAG, "LOCAL DEVICE: id: ${localDevice.id}, type: ${localDevice.deviceType},
manufacturer: ${localDevice.manufacturer}, model: ${localDevice.model}, name:
${localDevice.name}")

val devices = deviceManager.getDevices(DeviceGroup.WATCH)
devices.forEach { device ->

    Log.i(TAG, "DEVICE: id: ${device.id}, type: ${device.deviceType}, manufacturer:
${device.manufacturer}, model: ${device.model}, name: ${device.name}")
}
```

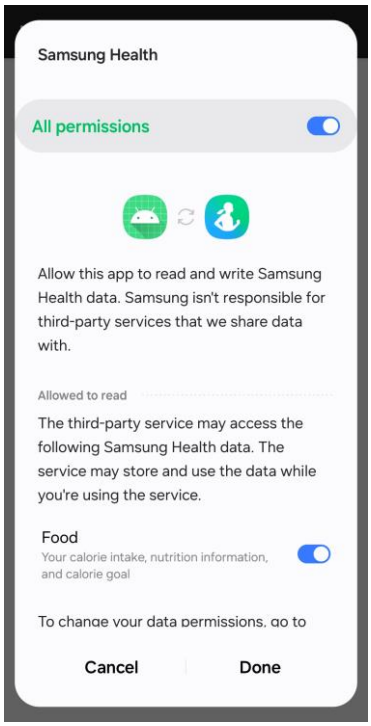
Sample Application – HealthDiary

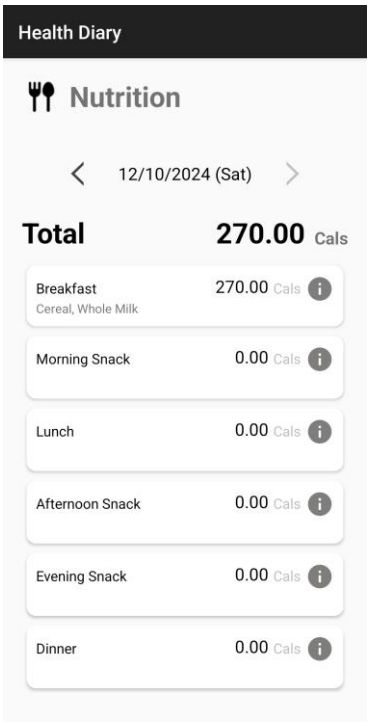
The HealthDiary application enables you to learn an entire data access flow. It includes the data permission request and representative data request operations. Requests for steps, sleep, nutrition, and heart rate data are handled on a daily basis.

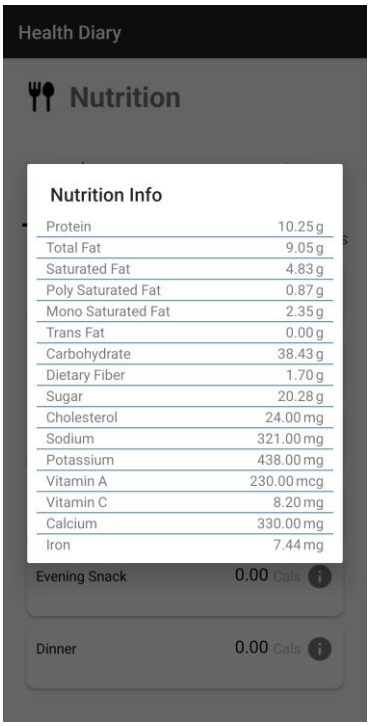
Open the HealthDiary application project and see the detailed code. If you build and run the HealthDiary application, the following main activity is shown.



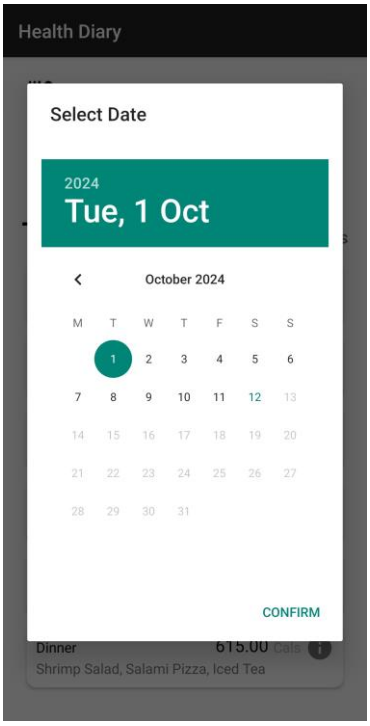
In the main activity display, select one of the data types. To acquire a data permission allowance, a data permission popup will be shown. If all permissions are allowed, the next page will show retrieved data from Samsung Health. Click on the information icon to get the nutrition information of the meal.

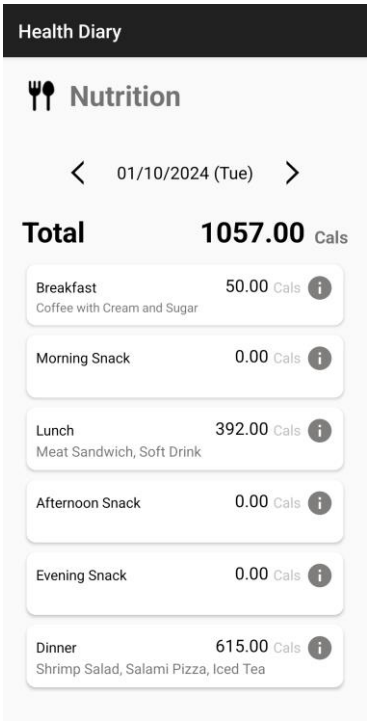




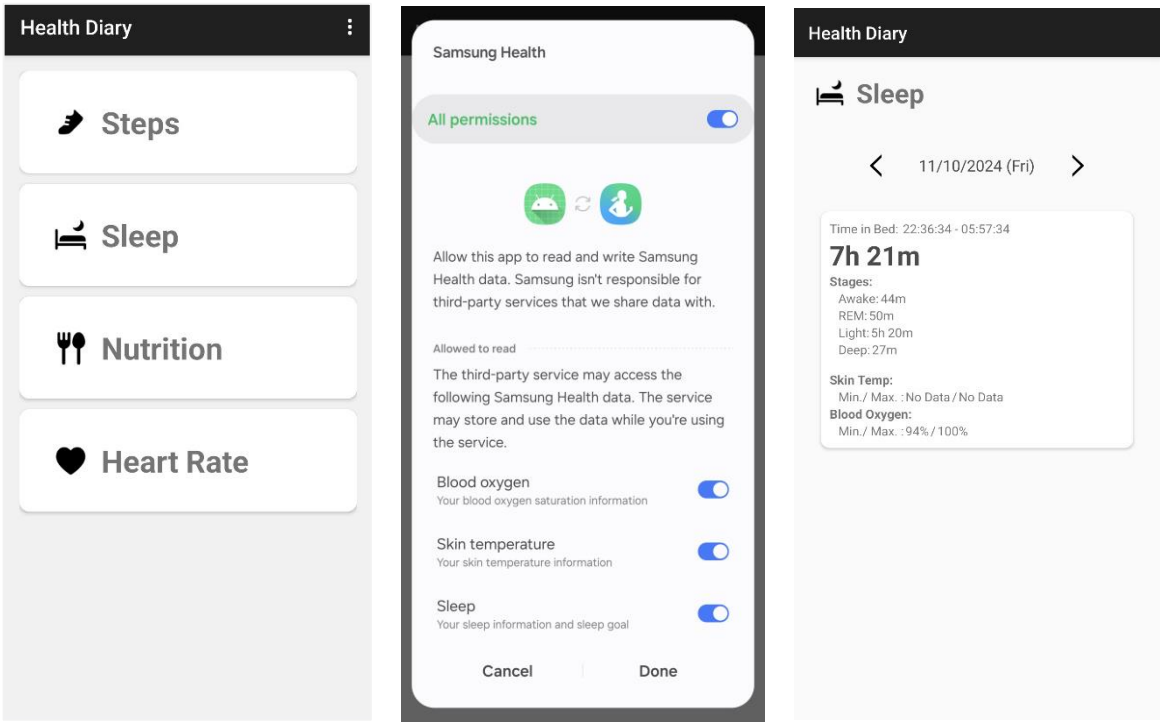


Changing to another date is available by selecting the date line. Whenever a date is changed, the sample application reads data for the given date and lists up the result data.





The sleep activity shows a reading associated data for a specific sleep data. The associated oxygen saturation and skin temperature are shown with an average aggregate operation.



Tool - DataViewer

The Samsung Health Data SDK provides the DataViewer tool to check the saved data easily in Samsung Health.

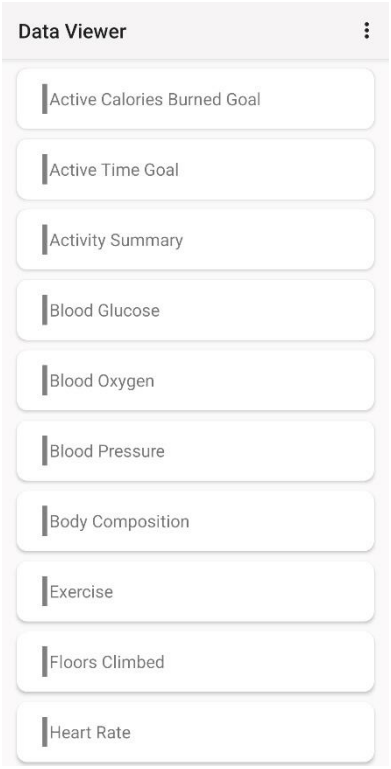
Install DataViewer

Find the DataViewer's apk file and install it onto your smartphone.

```
> adb install dataviewer.apk
```

Checking Saved Data

After installation, open the DataViewer, you can now check the saved data in Samsung Health by selecting one of the data types in the figures below:



Each data type can be accessed after getting a data permission allowance. Whenever you select a data type, the data permission popup displays. In the saved data list, you can see data details by selecting each data.